

Week 5: Implementation Reflection

Aaron Foster

Southern New Hampshire University

CS 660: Adv Prog Language Theory

Prof. Shadha Tabatabai

Aug 9th, 2026

This reflection addresses Part Two of the Module Five Activity. All three paradigm implementations are contained in the accompanying Python file, `module_five_paradigms.py`. The paradigm selected for detailed explanation is the functional approach.

Part Two, Item 4: How the Algorithm Works in the Functional Paradigm

Main Ideas

The functional implementation is built on three ideas: pure functions, immutability, and composition of higher-order functions.

The two rules that define the problem are written as standalone pure functions. `is_odd(number)` returns `True` when a value should be kept, and `square(number)` returns the value it contributes to the result. Both are pure: their output depends only on their argument, they read no external state, and they change nothing outside themselves. This gives them referential transparency any call to `square(3)` can be replaced by the literal `9` anywhere in the program without altering its meaning.

Those two rules are then handed to `filter` and `map`, which are higher-order functions: functions that accept other functions as arguments. This is the central move of the paradigm. Rather than writing instructions describing how to walk a list, I supply the behavior that applies to a single element and let a general-purpose combinator handle the traversal. The entire solution is one expression:

```
return list(map(square, filter(is_odd, numbers)))
```

The result is declarative. The line states what the output is: the squares of the odd elements rather than prescribing the sequence of steps that produces it. There is no loop counter, no accumulator, and no variable that holds different values at different points in time.

How Data Moves

Data moves in one direction through a pipeline and is never modified in place. The expression reads inside-out, following the flow:

- **numbers** enters as the source sequence. It is only ever read.
- **filter(is_odd, numbers)** produces a lazy iterator that yields only the elements for which the predicate returns True.
- **map(square, ...)** wraps that iterator and yields the square of each value it receives.
- **list(...)** drives the pipeline, pulling values through it and collecting them into a new list, which becomes the return value.

Because map and filter are lazy in Python 3, no intermediate list of odd numbers is ever constructed. Each element is pulled through the full pipeline before the next one is examined: the value 1 is tested, squared, and appended to the output before the value 2 is ever looked at. Memory holds one element in flight at a time rather than a complete intermediate collection.

The most important property of this flow is that the caller's list is untouched. Nothing in the pipeline mutates numbers; list() materializes an entirely new list. The demonstration in main() verifies this explicitly by confirming the input list is unchanged after all three implementations have run. This is the clearest contrast with the imperative version, where the

result variable takes on a different value with every iteration and the answer is whatever state that variable happens to hold when the loop exits. In the functional version there is one value that flows; in the imperative version there is one variable that changes.

The alternate implementation, `square_odds_functional_reduce`, makes the same flow explicit as a fold. Its accumulator expression, `acc + [square(n)]`, returns a brand new list on every step instead of mutating an existing one, the functional counterpart to calling `.append()`.

What Decisions the Code Makes

The algorithm makes exactly one decision per element, and it is made by `is_odd`:

```
return number % 2 != 0
```

A nonzero remainder on division by two means the value is odd and is kept. An even value is excluded and contributes nothing to the output — it is not replaced by a placeholder or a zero, it simply never reaches `map`. Squaring is not a decision at all: it is a total operation applied unconditionally to every value that survives the filter.

What is structurally interesting is where that decision lives. It is not an `if` statement embedded in the body of a loop. It is a value — a function object passed as an argument — and the branching itself is owned by `filter`. The decision is therefore a reusable, independently testable piece of the program rather than a line that only exists inside one loop.

Three edge cases confirm the behavior, and are exercised by the test cases in `main()`:

- **An empty list** yields an empty list. No decisions are made because the pipeline is never driven.

- **A list of all evens** yields an empty list. Every element is rejected by the predicate, so map receives nothing.
- **Negative odd numbers** are handled correctly. Python's modulo operator returns a non-negative result for a positive divisor, so `-3 % 2` evaluates to 1 and -3 is correctly identified as odd, producing 9. A test written as `number % 2 == 1` would be equally correct here but would silently fail in a language such as C or Java, where the remainder takes the sign of the dividend. Isolating the predicate as its own named function is what made this edge case easy to reason about and test in isolation.

Part Two, Item 5: How the Paradigm Influenced My Design Decisions

Writing the same algorithm three times made the influence of each paradigm visible, because each one pushed a different set of choices onto me.

What Gets a Name

The functional paradigm pressured me to name things the other two left anonymous. Because filter and map require functions as arguments, the two rules of the problem had to become `is_odd` and `square` — top-level, reusable, independently testable. In the imperative version the same two rules are buried inside the loop body as the condition `number % 2 != 0` and the expression `number ** 2`. They cannot be tested or reused without copying them. The object-oriented version also names them, as `should_keep` and `transform`, but binds them to a class: reusing that predicate elsewhere means instantiating an object first. Only the functional version produced rules that stand completely on their own.

Where State Is Allowed to Live

The imperative version required me to invent state that the problem never asked for. The result list does not appear in the problem statement; it exists purely as a mechanism, and having created it I then had to decide when to initialize it and when to mutate it. Its correctness depends on execution order. The functional paradigm gave me nowhere to put such a variable, so I wrote the solution as a single expression with no local variables at all. Ordering bugs are not avoided in that version so much as made structurally impossible, because there is no mutable state whose order of modification could be wrong.

Who Owns the Loop

In the imperative version I write the traversal myself and am responsible for every part of it. In the functional version, filter and map own the traversal and I only supply what happens to a single element. This removed an entire category of error from my code — wrong accumulator initialization, a forgotten append, an incorrect boundary — not through care on my part but because I never wrote the machinery where those errors occur. Notably, the object-oriented version arrives at a similar outcome: the process method contains the only traversal, and subclasses never touch it.

The Tradeoff I Would Weigh Differently

I want to be honest that functional was not the better choice on every axis. The object-oriented version is the most extensible of the three. Its abstract base class enforces a contract: any new transformer must supply both `should_keep` and `transform`, and a change in requirements — cubing even numbers, for instance — is a new subclass with the process method

untouched. The functional version handles that same change more cheaply, by passing different arguments, but it enforces nothing. Nothing stops a caller from supplying a predicate that returns a string. The object-oriented design front-loads structure and pays for it in ceremony; the functional design front-loads nothing and pays for it in guarantees. For a self-contained transformation like this one I would choose functional. For a codebase where many transformers must be written by different developers over time, I would choose the object-oriented design.

The functional version also carries a readability cost I had to accept. Composed calls nest inside-out, so `list(map(square, filter(is_odd, numbers)))` is read in the opposite direction from the way the data actually flows. The imperative version has no such gap between how it reads and what it does. I wrote the reduce variant partly to explore this, and it was instructive: the fold makes the traversal explicit but reintroduces the accumulator concept that the paradigm was helping me avoid, and it rebuilds the list on every step. I kept it as a secondary demonstration rather than the primary implementation.

Even the Comments Changed

One unexpected effect was that the in-line comments in Part One had to explain different kinds of things. The imperative comments are about timing, when state changes and in what order. The object-oriented comments are about responsibility, which class owns which decision and why. The functional comments are about identity, what each function is and what guarantees it offers, because in that version there is no meaningful "when" to describe. That difference is a fair summary of what separates the three paradigms.

Program Output

The screenshot below shows the output of `module_five_paradigms.py`, confirming that all three paradigm implementations produce identical results and that the input list is never mutated.

```
~/Desktop/COMP-SCI/MASTERS/CS 660/MOD 5
> python3 module_five_paradigms.py
CS 660 Module Five Activity - Paradigm Comparison
=====
Input list: [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]

1. Imperative result:      [1, 9, 25, 49, 81]
2. Object-oriented result: [1, 9, 25, 49, 81]
3. Functional result:      [1, 9, 25, 49, 81]
   Functional (reduce):    [1, 9, 25, 49, 81]

All paradigms produce identical output: True
Original list unchanged:                  True

Additional test cases
-----
[]          -> []
[2, 4, 6]   -> []
[1, 3, 5]   -> [1, 9, 25]
[-3, -2, 0, 7] -> [9, 49]

~/Desktop/COMP-SCI/MASTERS/CS 660/MOD 5
> 
```